

```

❶ def _create_alien(self, x_position):
    """Create an alien and place it in the row."""
    new_alien = Alien(self)
    new_alien.x = x_position
    new_alien.rect.x = x_position
    self.aliens.add(new_alien)

```

The method `_create_alien()` requires one parameter in addition to `self`: the `x`-value that specifies where the alien should be placed ❶. The code in the body of `_create_alien()` is the same code that was in `_create_fleet()`, except we use the parameter name `x_position` in place of `current_x`. This refactoring will make it easier to add new rows and create an entire fleet.

Adding Rows

To finish the fleet, we'll keep adding more rows until we run out of room. We'll use a nested loop—we'll wrap another `while` loop around the current one. The inner loop will place aliens horizontally in a row by focusing on the aliens' `x`-values. The outer loop will place aliens vertically by focusing on the `y`-values. We'll stop adding rows when we get near the bottom of the screen, leaving enough space for the ship and some room to start firing at the aliens.

Here's how to nest the two `while` loops in `_create_fleet()`:

```

def _create_fleet(self):
    """Create the fleet of aliens."""
    # Create an alien and keep adding aliens until there's no room left.
    # Spacing between aliens is one alien width and one alien height.
    alien = Alien(self)
    ❶ alien_width, alien_height = alien.rect.size

    ❷ current_x, current_y = alien_width, alien_height
    ❸ while current_y < (self.settings.screen_height - 3 * alien_height):
        while current_x < (self.settings.screen_width - 2 * alien_width):
            ❹ self._create_alien(current_x, current_y)
            current_x += 2 * alien_width

        ❺ # Finished a row; reset x value, and increment y value.
        current_x = alien_width
        current_y += 2 * alien_height

```

We'll need to know the alien's height in order to place rows, so we grab the alien's width and height using the `size` attribute of an alien `rect` ❶. A `rect`'s `size` attribute is a tuple containing its width and height.

Next, we set the initial `x`- and `y`-values for the placement of the first alien in the fleet ❷. We place it one alien width in from the left and one alien height down from the top. Then we define the `while` loop that controls how many rows are placed onto the screen ❸. As long as the `y`-value for the next row is less than the screen height, minus three alien heights, we'll keep adding rows. (If this doesn't leave the right amount of space, we can adjust it later.)